

AI-Powered Cybersecurity Scanning Platform — Full System Blueprint

Overview

Build a modern cybersecurity SaaS platform that provides:

1. Web Vulnerability Scanning
2. Network Vulnerability Scanning
3. Malware & Phishing Detection
4. AI-based Remediation Suggestions
5. Real-time Scan Progress Tracking
6. Interactive Security Dashboards
7. PDF/JSON Security Reports
8. Multi-user Authentication & Project Management

The platform should support enterprise-grade UI/UX with a cyberpunk-inspired dark interface, smooth animations, responsive design, and real-time scan updates.

Recommended Technology Stack

Frontend

Core

- ReactJS
- TypeScript
- Next.js 15
- TailwindCSS
- Framer Motion
- Shadcn/UI
- Recharts
- Axios
- Zustand or Redux Toolkit

Charts

- Recharts
- Chart.js

Authentication

- Clerk Authentication OR
 - Firebase Authentication
-

Backend

API Layer

- Node.js
- Express.js
- TypeScript

Database

- MongoDB Atlas

Real-Time Features

- Socket.IO

Queue System

- BullMQ + Redis

Security Scanner Engine

- Python FastAPI microservices
-

Scanner Engines

Web Scanner Engine

Recommended Tools

- OWASP ZAP API
- Nuclei
- Wapiti
- Nikto
- SQLMap (controlled mode)

Detectable Vulnerabilities

OWASP Top 10:

1. Broken Access Control
2. Cryptographic Failures
3. Injection
4. Insecure Design
5. Security Misconfiguration
6. Vulnerable Components
7. Authentication Failures

8. Software Integrity Failures
9. Logging Failures
10. SSRF

Additional:

- XSS
 - CSRF
 - Open Redirect
 - Directory Traversal
 - Exposed Admin Panels
 - Missing Security Headers
 - CORS Misconfiguration
 - Weak TLS/SSL
 - Sensitive Data Exposure
-

Network Scanner Engine

Recommended Tools

- Nmap
- RustScan
- Nessus Integration
- OpenVAS/GVM

Features

- Open Port Discovery
 - Service Detection
 - CVE Matching
 - OS Fingerprinting
 - Weak Services Detection
 - Outdated Software Detection
 - SMB Vulnerabilities
 - SSH Weak Configuration
 - FTP Anonymous Login Detection
-

Malware Scanner Engine

Recommended APIs & Engines

- VirusTotal API
- URLScan.io
- PhishTank
- AbuseIPDB
- Hybrid Analysis

Detectable Threats

- Phishing Domains
 - Malware Redirects
 - Trojan Delivery URLs
 - Malicious JavaScript
 - Suspicious iFrames
 - Crypto Miners
 - Drive-by Downloads
 - Browser Exploit Kits
-

Application Architecture

Frontend Architecture

```
src/  
├─ app/  
├─ components/  
│   ├─ dashboard/  
│   ├─ charts/  
│   ├─ scanner/  
│   ├─ tables/  
│   ├─ animations/  
│   └─ ui/  
├─ pages/  
├─ hooks/  
├─ lib/  
├─ services/  
├─ store/  
├─ types/  
└─ utils/
```

Main Dashboards

1. Web Dashboard

Scanning Card View

UI Components

- URL Input
- Scan Type Dropdown
- Start Scan Button

- Scan Progress Animation
- Live Status Messages

Scan Flow

```
graph TD; A[Enter URL] --> B[Validate URL]; B --> C[Start OWASP Scan]; C --> D[Queue Scan Job]; D --> E[Real-time Progress]; E --> F[Store Results]; F --> G[View Result Dashboard];
```

View Result Dashboard

Sections

A. Risk Exposure List

Table Columns

Vulnerability	Severity	CVSS Score
---------------	----------	------------

Severity Levels

- Informational
- Low
- Medium
- High
- Critical

B. Vulnerabilities Detected

Display:

- Total vulnerabilities found
- Severity breakdown

Example:

42 Vulnerabilities Found

C. Critical Vulnerabilities

Display:

7 Critical Issues

Condition:

- CVSS > 8

D. SAVE Score

Formula

$$\text{SAVE Score} = 10 - (\text{Total Risk Score} / \text{Maximum Risk}) * 10$$

Score Color Mapping

Score	Status
9-10	Excellent
7-8	Good
5-6	Moderate
3-4	Risky
0-2	Critical

E. Solutions Panel

Example

Vulnerability	Solution
XSS	Sanitize user input
SQL Injection	Use prepared statements
Missing CSP	Add CSP headers

F. Pie Chart

Chart Categories

- Critical
- High
- Medium
- Low
- Informational

Recommended Library

- Recharts PieChart
-

2. Network Dashboard

Scanner Installation Flow

```
Download Scanner Agent
↓
Install Agent
↓
Login
↓
Choose Scan Mode
↓
Generate UUID
```

Network Agent

Recommended Language

- GoLang OR
- Rust

Why?

- Lightweight
 - Fast scanning
 - Cross-platform support
 - Secure networking
-

Agent Responsibilities

- Local network discovery
 - Port scanning
 - Service enumeration
 - System fingerprinting
 - Secure communication with backend
 - UUID registration
-

Network Dashboard UI

Input

- UUID Input Field
- Scan Button

Result Sections

A. Risk Exposure Table

Vulnerability	Severity	CVSS
---------------	----------	------

B. Vulnerabilities Detected

28 Vulnerabilities Found

C. Critical Vulnerabilities

4 Critical Risks

D. SAVE Score

Animated circular score indicator.

E. Open Ports Table

Port	Service	Risk
22	SSH	Medium
445	SMB	Critical
3389	RDP	High

F. Pie Chart

Visual distribution of:

- Open Ports
- Vulnerabilities
- Severity Levels

3. Malware Dashboard

Flow

```
graph TD; A[Enter Domain] --> B[Initialize Threat Intelligence APIs]; B --> C[Fetch Domain Reputation]; C --> D[Analyze Redirects]; D --> E[Analyze Scripts]; E --> F[Generate Malware Report];
```

Malware Dashboard Views

A. Scanning Card

Inputs

- Domain Name
- Scan Button

B. Live Scanning View

Real-time Updates

Checking DNS Reputation...
Analyzing JavaScript...
Checking Redirect Chains...
Analyzing SSL Certificate...
Detecting Phishing Patterns...

UI Features

- Animated terminal logs
 - Progress bars
 - Neon scanning animation
 - Threat radar effect
-

C. Report View

Sections

Threat Level

Level	Meaning
Safe	No threat
Suspicious	Requires review
Dangerous	Malware/phishing detected

Malware Indicators

- Suspicious redirects
 - Hidden iframes
 - Malicious scripts
 - Fake login forms
 - Blacklisted IPs
-

Graphs

Threat Timeline

Malware Categories

Reputation Score

Backend API Structure

Authentication

```
POST /api/auth/login  
POST /api/auth/register
```

Web Scanner

```
POST /api/web/scan  
GET /api/web/result/:id  
GET /api/web/history
```

Network Scanner

```
POST /api/network/scan  
GET /api/network/result/:id  
POST /api/network/register-agent
```

Malware Scanner

```
POST /api/malware/scan  
GET /api/malware/result/:id
```

Database Collections

Users

```
{
  "name": "",
  "email": "",
  "password": "",
  "role": "user"
}
```

Scans

```
{
  "userId": "",
  "scanType": "web",
  "target": "",
  "status": "completed",
  "saveScore": 7.5,
  "createdAt": ""
}
```

Vulnerabilities

```
{
  "scanId": "",
  "name": "SQL Injection",
  "severity": "critical",
  "cvss": 9.1,
  "description": "",
  "solution": ""
}
```

Real-Time Scan Progress

Using Socket.IO

Events

```
scan:start  
scan:progress  
scan:completed  
scan:error
```

AI Features

AI Security Assistant

Integrate:

- OpenAI API
- Claude API
- Gemini API

Capabilities

- Explain vulnerabilities
 - Generate remediation steps
 - Security recommendations
 - Risk summaries
 - Threat prioritization
-

Security Requirements

Must Implement

- JWT Authentication
 - Rate Limiting
 - CSRF Protection
 - Helmet.js
 - Input Validation
 - API Key Encryption
 - Secure File Handling
 - HTTPS Enforcement
 - Audit Logging
-

UI/UX Design Recommendations

Theme

Cyberpunk + Enterprise Security Dashboard

Colors

- Black background
 - Neon cyan
 - Purple accents
 - Red critical alerts
 - Glassmorphism cards
-

Animations

Recommended Effects

- Matrix rain loader
 - Pulse scan lines
 - Radar sweep
 - Typing terminal logs
 - Animated charts
 - Glowing borders
 - Live activity indicators
-

Deployment Architecture

Frontend

- Vercel

Backend

- Railway OR
- Render OR
- AWS ECS

Database

- MongoDB Atlas

Redis

- Upstash Redis

Scanner Engines

- Dedicated VPS
 - Docker containers
-

Docker Architecture

Services

```
frontend  
backend  
scanner-engine  
redis  
mongodb  
worker  
socket-server
```

Suggested Development Phases

Phase 1

- Authentication
- Dashboard UI
- Web Scanner Integration
- Result Pages

Phase 2

- Network Scanner Agent
- UUID System
- Open Port Analysis

Phase 3

- Malware Detection
- Threat Intelligence APIs
- Live Scan Logs

Phase 4

- AI Security Assistant
- PDF Reports
- Team Collaboration

Phase 5

- Enterprise Features
- Multi-tenant Architecture
- Billing System
- Subscription Plans

Recommended Folder Structure

```
project/  
├─ frontend/  
├─ backend/  
├─ scanner-engine/  
├─ agent/  
├─ docker/  
└─ docs/
```

Important Legal Considerations

Only allow users to scan:

- Their own websites
- Systems they own
- Systems they are authorized to test

Add:

- Terms & Conditions
 - Authorization agreement
 - Logging and monitoring
 - Abuse prevention
-

Best Development Order

Recommended Start

1. Frontend UI
 2. Authentication
 3. Web scanner API
 4. Result dashboard
 5. Socket.IO real-time updates
 6. Network scanner agent
 7. Malware analysis
 8. AI remediation engine
 9. PDF reports
 10. Production optimization
-

Recommended Extra Features

Enterprise Features

- Team workspace
 - Scheduled scans
 - CI/CD integrations
 - Slack/Discord alerts
 - Jira integration
 - GitHub Security sync
 - Scan history analytics
 - Export to PDF/CSV
 - Dark/light themes
 - RBAC permissions
-

Suggested Branding Ideas

Possible Names

- VulnX
 - CyberPulse
 - SentinelAI
 - ThreatScope
 - BlackShield
 - HexaSecure
 - NetSpectre
 - PhantomGuard
-

Final Recommendation

The best architecture for this project:

Frontend:

- Next.js + TypeScript + Tailwind + Framer Motion

Backend:

- Node.js + Express + Socket.IO + Redis

Scanner Layer:

- Python FastAPI + OWASP ZAP + Nmap + VirusTotal APIs

Database:

- MongoDB Atlas

Deployment:

- Vercel + AWS/VPS

This stack provides:

- Scalability
- Real-time scanning
- Professional UI/UX
- Enterprise-grade architecture
- Easy future expansion
- High performance
- Secure infrastructure